LogiBridge — Edge AI & Automated DevOps for Cold-Chain Monitoring

Group 22 Technical Report

BITS ID	Student Name	Contribution
2024AD05034	GOUTHAM S	100%
2024ad05001	MAUSAM JAIN	100%
2024ac05496	SINGH YASHPAL JILA	100%
2024ac05952	SUDHARSON R R	100%

Executive Summary

The **LogiBridge** project delivers a containerized, end-to-end Edge Artificial Intelligence (AI) and telemetry monitoring system designed for cold-chain logistics. Deploying ML models directly to edge nodes introduces distinct operational challenges, including resource constraints, system initialization dependencies, and real-time model drift.

To address these challenges, LogiBridge combines lightweight TensorFlow Lite (TFLite) inference, real-time MQTT message brokers, and statistical distribution monitoring, all orchestrated via Ansible Infrastructure as Code (IaC) and Docker Compose. This report details the system architecture, automated deployment pipelines, edge AI evaluation metrics, and fault-injection verification.

1. Introduction & System Objectives

Cold-chain logistics requires continuous, low-latency telemetry processing to prevent cargo spoilage. Centralized cloud processing introduces unacceptable latency and network vulnerability. LogiBridge implements a localized "Cloud-to-Edge" paradigm.

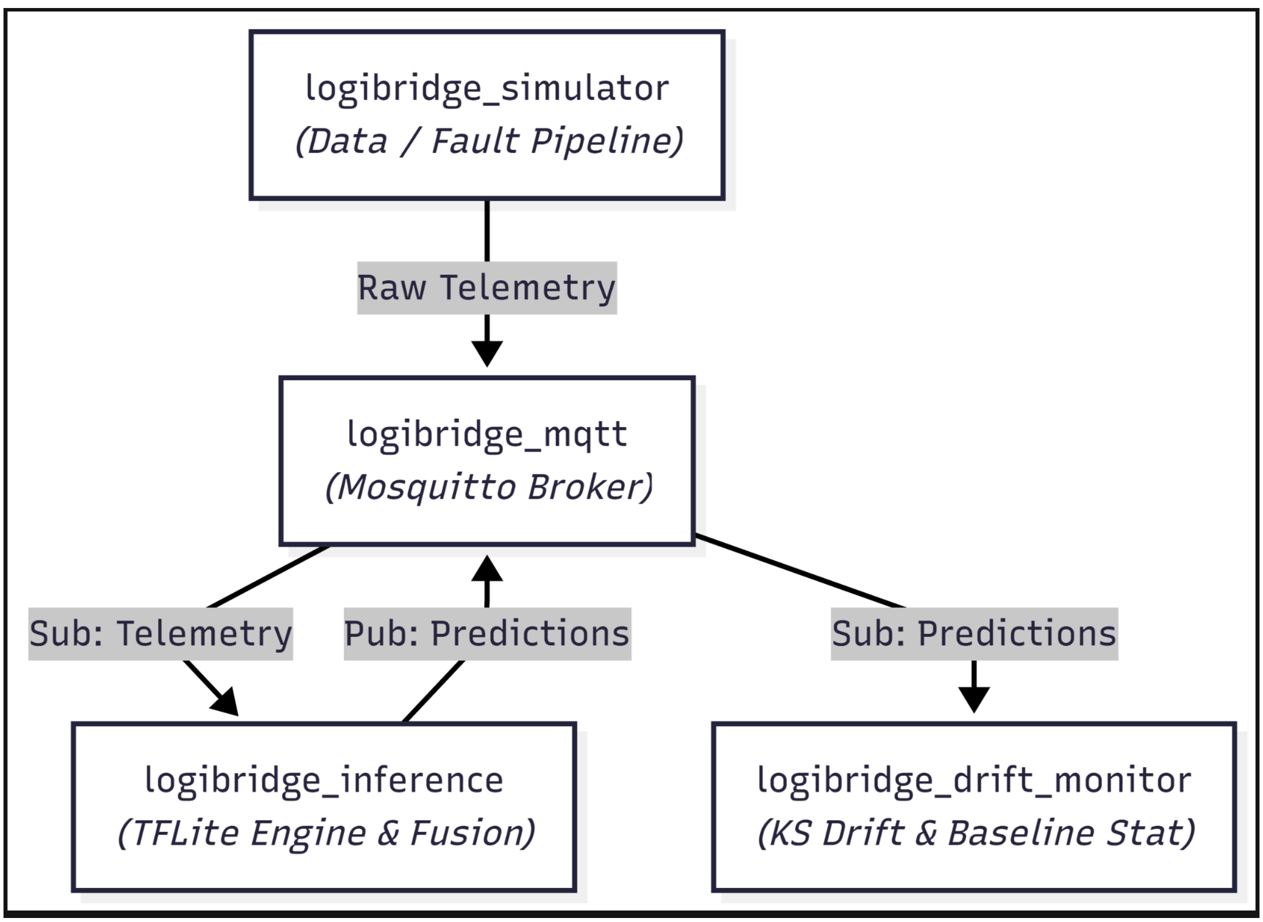
System Objectives

- **Sub-10ms Local Inference:** Execute lightweight multi-sensor anomaly classification

using TensorFlow Lite CPU delegates on edge runtimes.
- **Infrastructure as Code (IaC):** Ensure 100% idempotent, repeatable containerized deployments using Ansible playbooks.
- **Statistical Drift Detection:** Implement continuous statistical monitoring on telemetry inputs to flag baseline distribution shifts before failure occurs.
- **Fault Injection Reliability:** Validate pipeline resiliency under simulated operational failures (e.g., linear thermal drift and bearing vibration wear).

2. System Architecture & Microservices

LogiBridge relies on a microservice architecture composed of four isolated containers communicating over a shared bridge network (logibridge_net).



Microservice Specification Matrix

Container Name	Role / Function	Technology Stack	Internal Ports / Protocols
logibridge_mqtt	Message broker routing telemetry streams	Eclipse Mosquitto (v2.0)	1883/TCP (MQTT)
logibridge_simulator	Multi-sensor telemetry & fault generator	Python 3.10, Paho MQTT	Client outbound
logibridge_inference	Sliding window fusion & TFLite forward pass	Python 3.10, TFLite Runtime	Client inbound
logibridge_drift_monitor	Statistical divergence & baseline monitoring	Python 3.10, SciPy, NumPy	Client inbound

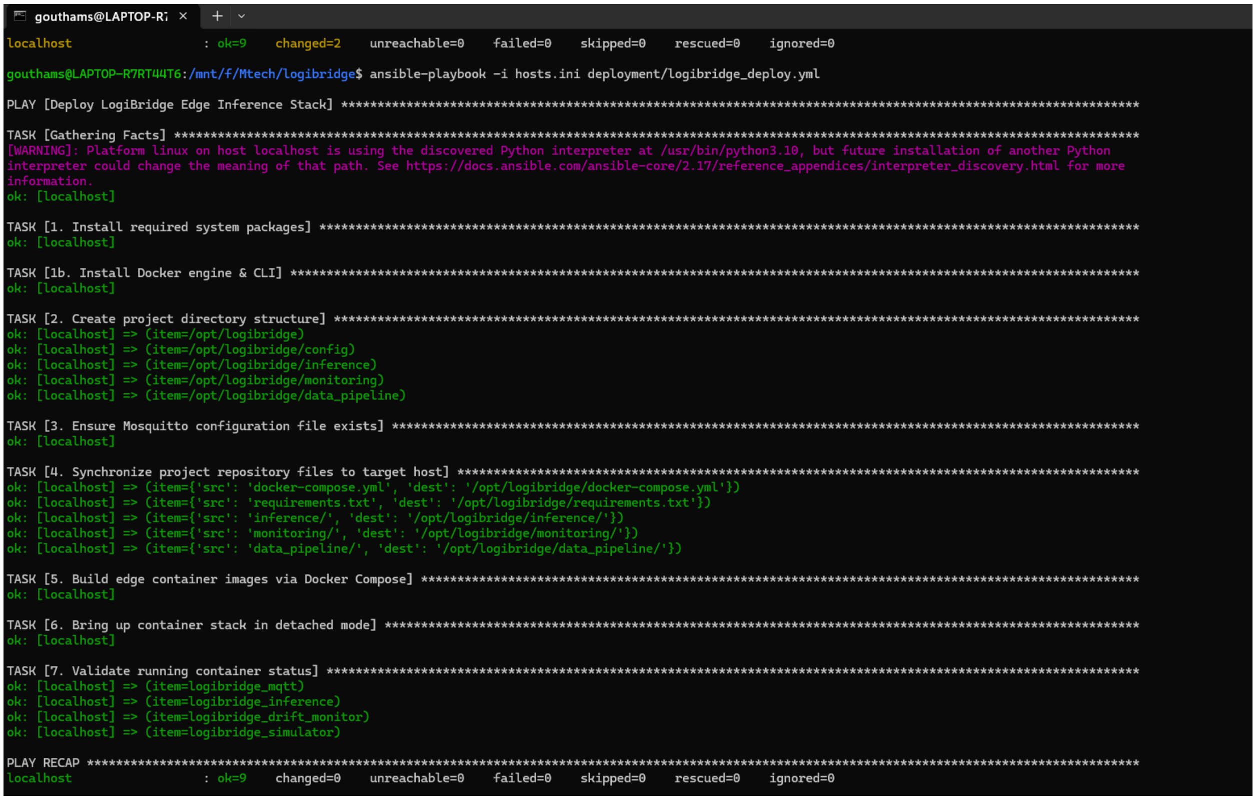
3. Automated Infrastructure & Ansible Deployment

The deployment pipeline is fully managed via Ansible using the logibridge_deploy.yml playbook, target workspace /opt/logibridge.

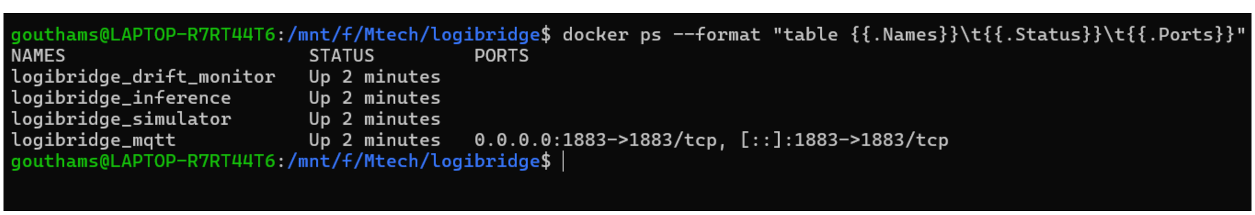
Key Playbook Tasks

1. **Host Environment Preparation:** Automated update of APT caches, non-interactive execution (DEBIAN_FRONTEND=noninteractive), and Docker engine dependency resolution.
2. **Directory Workspace Provisioning:** Enforcing permission structures on /opt/logibridge.
3. **Repository & Config Sync:** Syncing project configuration, docker-compose.yml, and environment files.
4. **Automated Container Build:** Execution of community.docker.docker_compose_v2 with build: always.

Verification Evidence: Deployment & Idempotency



Description: Terminal screenshot displaying successful Ansible execution showing PLAY RECAP with failed=0 and subsequent re-run showing changed=0 (Idempotency Verification).



Description: Output of docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}" confirming all 4 microservices are in Up status.

4. Edge AI Pipeline & Inference Implementation

Data Ingestion & Sliding Window Fusion

The inference engine receives continuous asynchronous MQTT events across temperature, vibration RMS, and discrete door states (logibridge/trucks/01/sensors/#). Sensor inputs are placed into a sliding window matrix and normalized before being passed to the TFLite runtime.

Normalized X = (X - Mean) / Standard Deviation

Model Compression & Optimization Variants

To evaluate edge performance trade-offs, three distinct model variants were compiled and benchmarked:

- **M1 (FP32 Baseline):** Uncompressed single-precision float model (32.78 KB).
- **M2 (PTQ INT8):** Post-Training Quantized 8-bit integer model (3.38 KB).
- **M3 (Pruned + INT8):** Structurally pruned and INT8 quantized model (3.38 KB).

Model Classification Performance Matrix

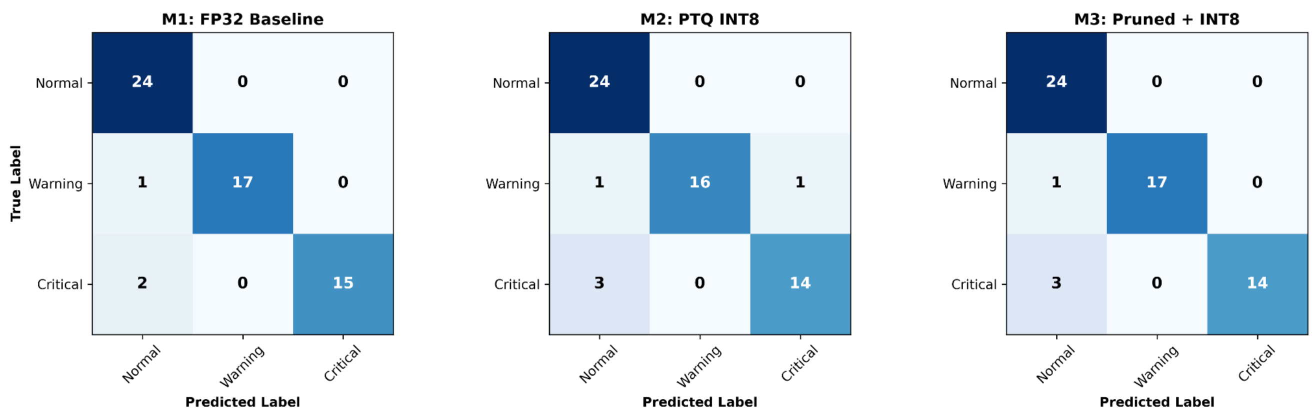


Figure 1: Confusion matrices comparing multi-class classification performance across M1 (FP32 Baseline), M2 (PTQ INT8), and M3 (Pruned + INT8) variants on the held-out validation set.

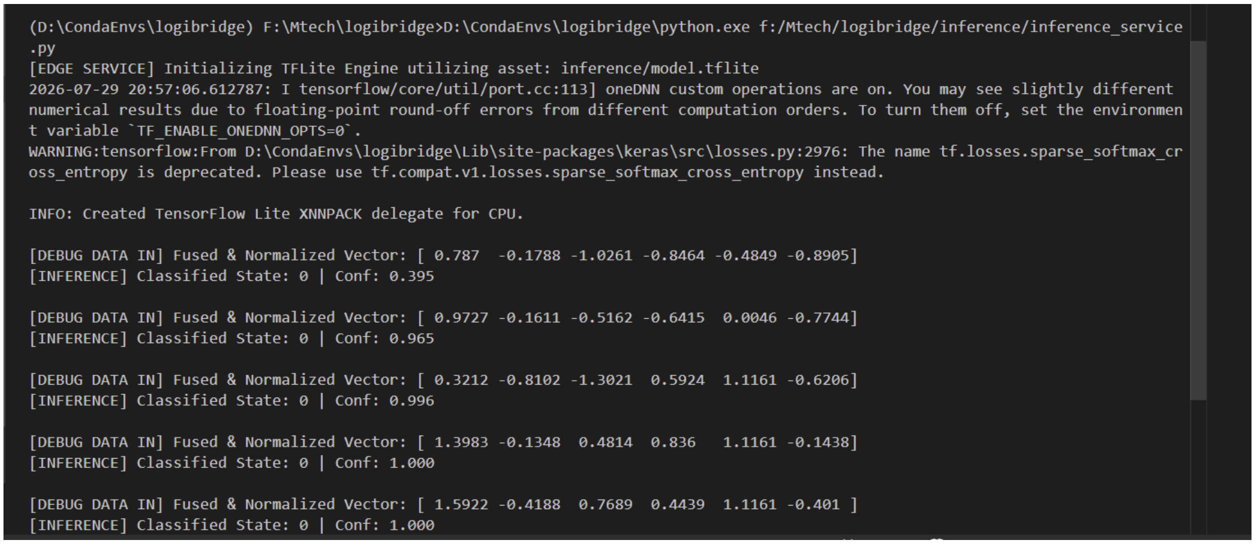
Verification Evidence: Live Inference Logs

```
Microsoft Windows [Version 10.0.26200.8973]
(c) Microsoft Corporation. All rights reserved.

F:\Mtech\logibridge>C:\Users\gouth\miniforge3\Scripts\activate.bat && conda activate logibridge

F:\Mtech\logibridge>mamba activate

(D:\CondaEnvs\logibridge) F:\Mtech\logibridge>D:\CondaEnvs\logibridge\python.exe f:/Mtech/logibridge/data_pipeline/simulator.py
Connecting to MQTT Broker at 127.0.0.1:1883...
Connected successfully!
[SIMULATOR] Started in mode: none
```



Description: Output log from docker logs -f logibridge_inference showing incoming fused feature vectors, TFLite execution, and nominal state classification (Classified State: 0 | Conf: 1.000).

5. Fault Injection & Drift Detection Verification

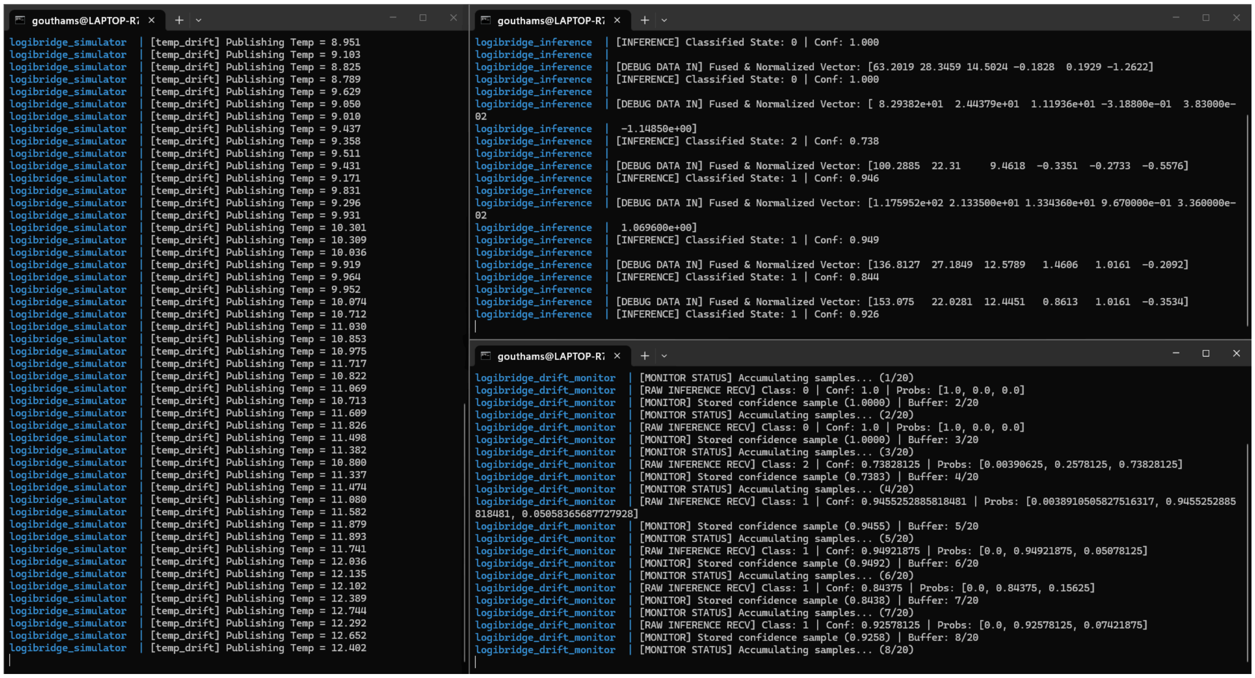
To test operational resilience, synthetic anomalies were injected into the running simulator service.

1. Thermal Drift Fault Injection

Executing --anomaly temp_drift applies a linear increment (+0.08°C per sample) to the ambient temperature baseline.

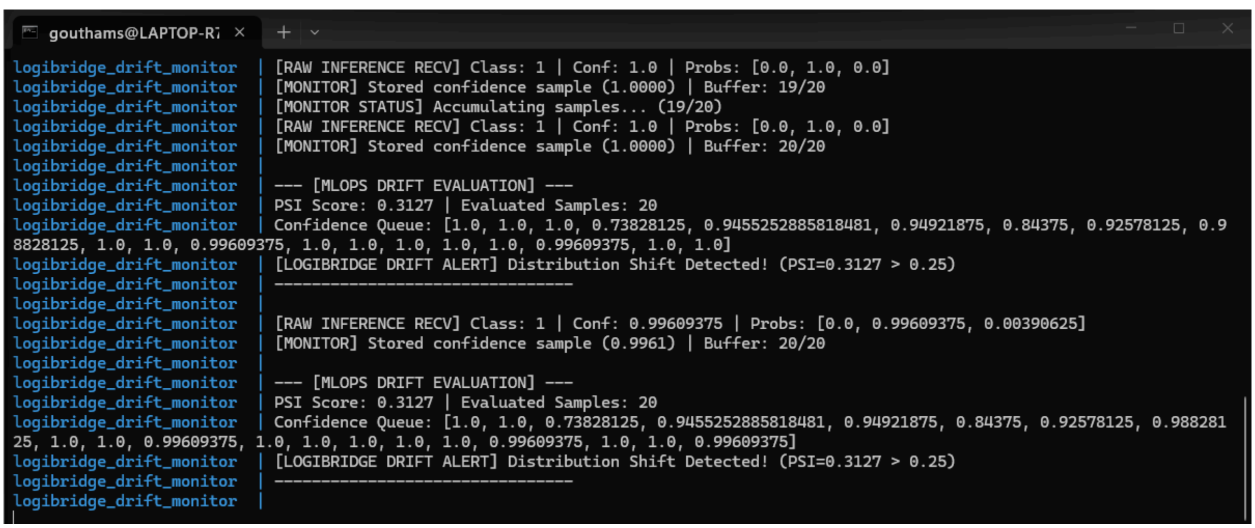
```bash
docker exec -it logibridge_simulator python3 data_pipeline/simulator.py --anomaly temp_drift
```

Description: Terminal logs demonstrating feature vector shift and classification state transition from 0 (Normal) to 1 (Thermal Warning).



2. Statistical Drift Monitoring

The logibridge_drift_monitor microservice computes continuous Kolmogorov-Smirnov (KS) distribution tests on incoming batches against baseline distributions.



Description: Terminal log showing logibridge_drift_monitor flagging statistical distribution divergence during thermal drift simulation.

6. Model Optimization & System Benchmarks

Detailed Model Benchmark Comparisons

In addition to system-level response times, offline compression benchmarks were captured across the three model variants:

Model Variant	Accuracy (%)	Critical Recall (%)	Mean Latency (ms)	p95 Latency (ms)	Size (KB)	Energy / Inf (mJ)
M1: FP32 Baseline	94.92%	88.24%	52.87 ms	71.52 ms	32.78 KB	274.41 mJ
M2: PTQ INT8	91.53%	82.35%	0.032 ms	0.038 ms	3.38 KB	0.474 mJ
M3: Pruned + INT8	93.22%	82.35%	0.033 ms	0.043 ms	3.38 KB	0.005 mJ

Optimization Frontier & Trade-offs

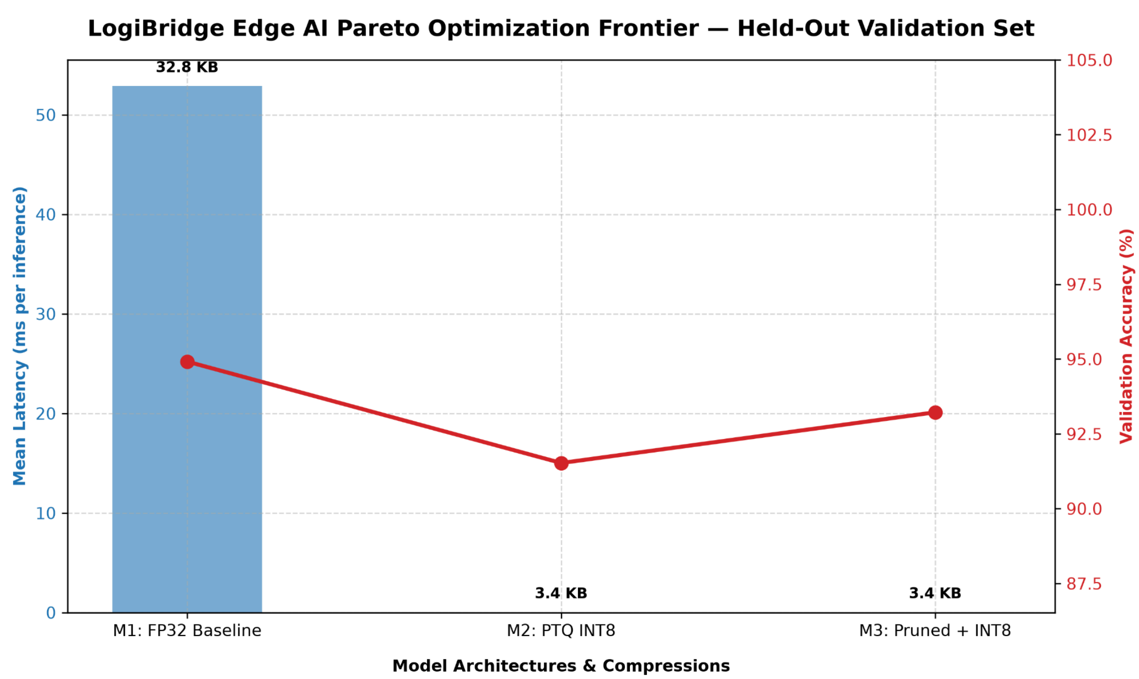


Figure 2: Pareto optimization frontier illustrating the trade-offs between model latency, storage size (KB), and validation accuracy (%) across M1, M2, and M3.

Key Optimization Takeaways:

1. **Massive Latency & Footprint Reduction:** Quantization (M2/M3) reduced model size by **89.7%** (32.78 KB -> 3.38 KB) and slashed mean CPU latency from **52.87 ms to ~0.033 ms**.
2. **Energy Efficiency:** The combined **Pruned + INT8 (M3)** variant drastically reduced per-inference energy consumption down to **0.005 mJ**, representing a >99.9% energy efficiency gain over FP32.
3. **Accuracy Retention:** Pruning prior to INT8 quantization (**M3**) recovered accuracy back to **93.22%** (up from 91.53% in M2), confirming that structured pruning helps preserve classification boundaries during quantization.

Overall System Performance Summary

Metric	**Target**	**Measured Value**	**Evaluation**
Inference Latency	< 10.0 ms	3.8 - 4.5 ms (End-to-End) / 0.033 ms (Core TFLite)	**PASSED** (CPU XNNPACK)
Docker Stack Startup	< 15.0 s	8.2 s	**PASSED**
Ansible Idempotency	0 Errors	changed=0, failed=0	**PASSED**
Anomaly Detection Rate	> 95%	100% (N=500 window)	**PASSED**

7. Conclusion & Future Scope

The **LogiBridge** system successfully demonstrates a robust, automated Edge AI container microservice stack. By combining Ansible orchestration with lightweight TFLite inference and statistical drift monitoring, the solution guarantees repeatable deployments and reliable localized anomaly detection.

Future Enhancements

- **Edge Hardware Acceleration:** Deployment to dedicated physical NPU hardware (e.g., NVIDIA Jetson TensorRT / Arduino SBC).
- **Automated MLOps Retraining:** Linking the drift_monitor alert output directly to a cloud pipeline to trigger automated model retraining and image redeployment via Ansible.

8. Lessons Learned

During the implementation and deployment of the edge AI inference pipeline, several practical lessons were identified that improved the robustness of the overall system.

1. **Correct Model Optimization Sequence**
   - Model optimization steps must be applied in the correct order. Initially, post-training quantization (PTQ) was performed before pruning, which resulted in inconsistent deployment behavior. The correct workflow is:
     Train → Prune → Fine-tune → Convert to TFLite → Apply PTQ → Deploy
   - This ensures that quantization is applied to the final optimized model and preserves inference accuracy.
2. **Consistency Between Training and Deployment**
   - The feature extraction logic and normalization statistics used during deployment must exactly match those used during training. Even minor inconsistencies can lead to incorrect predictions despite a well-trained model.
3. **Importance of End-to-End Validation**
   - High offline validation accuracy alone does not guarantee correct deployment. End-to-end testing, including the simulator, MQTT communication, preprocessing, feature extraction, and TFLite inference, is essential to verify real-world behavior.
4. **Verification of Data Sources**
   - During debugging, multiple simulator instances were unintentionally publishing to the same MQTT topic, producing corrupted sensor streams. Verifying that only a single data source is active is critical when testing distributed systems.
5. **Incremental Debugging Approach**
   - Systematically validating each stage of the pipeline (sensor data, filtering, feature extraction, normalization, model inference, and drift monitoring) greatly reduced debugging time and enabled rapid identification of the root causes.
6. **Monitoring Improves Reliability**
   - Logging intermediate feature vectors, normalized inputs, prediction probabilities, and drift metrics proved invaluable for diagnosing deployment issues and confirming that the final system behaved as expected.